**** Static Synchronization :**

If we make our synchronized method as static then it is called static synchronization.

For static synchronization "Object is not required". Static method we can call with the help of class name.

In static synchronization, the thread will take the lock from the class but not from the Object. [Every class has a lock]

Unlike objects, We can't create multiple classes in the same package.

//Programs :

```
package com.ravi.static_synchronization;

class Table
{
    public static synchronized void printTable(int num)
    {
        for(int i=1; i<=10; i++)
        {
            IO.println(num+" X "+i+" = "+(num*i));
            try
            {
                Thread.sleep(1000);
            }
            catch(InterruptedException e)
            {
                e.printStackTrace();
            }
        }
        IO.println("............");
    }
}

public class StaticSynchronizationDemo {

    public static void main(String[] args)
    {
        Thread t1 = new Thread()
        {
          @Override
          public void run()
          {
              Table.printTable(5);
          }
        };

        Thread t2 = new Thread()
        {
          @Override
          public void run()
          {
              Table.printTable(10);
          }
        };

        Thread t3 = new Thread()
        {
          @Override
          public void run()
          {
              Table.printTable(15);
          }
        };

        Thread t4 = new Thread()
        {
          @Override
          public void run()
          {
              Table.printTable(20);
          }
        };

        t1.start();  t2.start();  t3.start();  t4.start();
    }

}
```

How to work with synchronized block, If we want to take the lock from the class.

```
synchronized(MyClass.class) //Taking the lock from the class
{

}
```

Thread Priority :

It is possible in java to assign priority to a Thread. Thread class has provided two predefined methods setPriority(int newPriority) and getPriority() to set and get the priority of the thread respectively.

```
class Thread implements Runnable
{
  private volatile int priority;

  public final void setPriority(int newPriority)
  {
    this.priority = newPriority;
  }

  public final int getPriority()
  {
    return this.priority.
  }
}
```

In java we can set the priority of the Thread in numbers from 1- 10 only where 1 is the minimum priority and 10 is the maximum priority.

Whenever we create a thread in java by default its priority would be 5 that is normal priority.

The user-defined thread created as a part of main thread will acquire the same priority of main Thread.

Thread class has also provided 3 final static fields which are as follows :-

```
public static final int MIN_PRIORITY = 1;
public static final int NORM_PRIORITY = 5;
public static final int MAX_PRIORITY = 10;
```

Note :- We can't set the priority of the Thread beyond the limit(1-10) so if we set the priority beyond the limit (1 to 10) then it will generate an exception java.lang.IllegalArgumentException.

```
package com.ravi.priority;

class Demo extends Thread
{
    @Override
    public void run()
    {
        int priority = Thread.currentThread().getPriority();
        IO.println("Child Thread priority is :"+priority);
    }
}

public class PriorityDemo1 {

    public static void main(String[] args)
    {
        Thread t1 = new Thread();
        IO.println("Priority of the thread is :"+t1.getPriority());

        Demo d1 = new Demo();
        d1.start();

    }

}
```

The default priority of a thread is 5

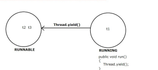
```
package com.ravi.priority;

class Test extends Thread
{
    @Override
    public void run()
    {
        int priority = Thread.currentThread().getPriority();
        IO.println("Child Thread priority is :"+priority);
    }
}

public class PriorityDemo2
{
   public static void main(String[] args)
   {
      Thread thread = Thread.currentThread();
      thread.setPriority(Thread.MIN_PRIORITY);
      //thread.setPriority(11); //java.lang.IleegalArgumentException
      IO.println("Main Thread priority is :"+thread.getPriority());

      new Test().start();

   }
}
```

Child Thread priority depends upon main thread

```
package com.ravi.priority;

class Foo extends Thread
{
    @Override
    public void run()
    {
        int count = 0;

        for(int i=1; i<=1000000; i++)
        {
           count++;
        }

        Thread t = Thread.currentThread();
        IO.println("Completed thread name is "+t.getName()+" and completed thread priority is
:"+t.getPriority());

    }
}

public class PriorityDemo3
{
    public static void main(String[] args)
    {
        Foo f1 = new Foo();
        Foo f2 = new Foo();

        f1.setName("Low_Priority");
        f2.setName("High_Priority");

        f1.setPriority(Thread.MIN_PRIORITY);
        f2.setPriority(Thread.MAX_PRIORITY);

        f1.start();  f2.start();
    }

}
```

Most of time the thread having highest priority will complete its task but we can't say that it will always complete its task first that means Thread schedular dominates Priority of the Thread.

Thread.yield() : [To Prevent a thread from over utilization of CPU]



It is a static method of Thread class.

The currently executing thread will send a notification to thread schedular to stop the currently executing Thread (In Running state) and provide a chance to Threads which are in Runnable state to enter inside the running state having same priority or higher priority than currently executing Thread.

Here The running Thread will directly move from Running state to Runnable state.

The Thread schedular may accept OR ignore this notification message given by currently executing Thread.

Here there is no guarantee that after using yield() method the running Thread will move to Runnable state and the thread which is running thread will move to Runnable state.

If the thread which is in runnable state is having low priority than the current executing thread in Running state,Running thread will continue its execution.

*It is mainly used to avoid the over-utilisation a CPU by the current Thread so all the threads will get equal chance of CPU
for execution as well as In industry we can use testing and debugging purpose

//Program

```
package com.ravi.yield;

class Test implements Runnable
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();

        for(int i=1; i<=10; i++)
        {
           IO.println("i value is :"+i+ " by "+name+" thread");

           if(name.equals("Child1"))
           {
               Thread.yield(); //given the chance to Child2
           }

        }

    }

}

public class YieldDemoExample
{
    public static void main(String[] args)
    {
       Test t1 = new Test();

        Thread thread1 = new Thread(t1, "Child1");
        Thread thread2 = new Thread(t1, "Child2");

        thread1.start();
        thread2.start();

    }
}
```

A thread is well known for independent execution, During the life cycle of a thread it passes through different states

Java has provided a new Thread life cycle from JDK 1.5V with the support of enum called Thread.State.

We have a predefined enum called State which is available inside Thread class and it represents life cycle constants which are as follows :

1) NEW (Born State)
2) RUNNABLE (Ready to Run)
3) BLOCKED (Waiting for Object lock to enter OR reenter in syn area)
4) WAITING (Waiting for another thread to complete, join() wait())
5) TIMED_WAITING (Waiting for time period to complete, sleep(1000))
6) TERMINATED (Dead)

NEW :

Whenever we create a thread instance(Thread Object) a thread comes to new state OR born state. New state does not mean that the Thread has started yet only the object or instance of Thread has been created.

RUNNABLE :

Whenever we call start() method on thread object, A thread moves to Runnable state i.e Ready to run state. Here the thread is considered "alive," but it doesn't immediately start execution unless the CPU scheduler assigns it time.

BLOCKED :

If a thread is waiting for object lock OR monitor to enter inside synchronized area OR re-enter inside synchronized area then it is in blocked state.

WAITING :

A thread in the waiting state is waiting for another thread to
perform a particular action but WITHOUT ANY TIMEOUT period.
A thread that has called wait() method on an object is waiting for another thread to call notify() or notifyAll() on the same object OR A thread that has called join() method is waiting for a specified thread to terminate.

TIMED_WAITING :

A thread in the timed_waiting state, if we call any method which put the thread into temporarly timed_waiting state but WITH POSITIVE TIMEOUT period like sleep(lons ms), join(long ms), wait(long ms) then the Thread is considered as Timed_Waiting state.

TERMINATED :

The thread has successuflly completed it's execution in the separate stack memory that means run() method is successfully completed.

==

****** Inter Thread Communication (ITC)**